

High Throughput Hardware Architecture for (1440,1344) Low-Density Parity-Check Code Utilizing Quasi-Cyclic Structure

Hiroyuki Yamagishi and Makoto Noda

Core Device Development Group, Sony Corporation

5-1-12 Kitashinagawa Shinagawa-ku, Tokyo, 141-0001 Japan

Email: Hiroyuki.Yamagishi@jp.sony.com

Abstract—High throughput architecture of an encoder and a decoder for a quasi-cyclic low-density parity-check (LDPC) code is proposed. A new systematic encoding method is carried out by polynomial manipulation. The proposed decoder architecture, where the check-node process is split into two processes so that the memory access becomes column-wise, enables overlapped message-passing for any parity-check matrix. The hardware architecture for the check-node processes utilizing a quasi-cyclic structure does not require complex multiplexers. Hardware employing the proposed architecture for a (1440,1344) LDPC code designed for high throughput millimeter wave application is evaluated using 65 nm CMOS technology. The gate count of the encoder for 3 Gbps and 6 Gbps throughput is 2.5 k and 3.1 k, respectively, and the gate count of the decoder for 8 iterations is 304 k and 409 k, respectively. A bit-error rate of 10^{-6} is obtained at E_b/N_0 of 5.9 dB, and the estimated power consumption of the decoder is 58 mW for 3 Gbps and 86 mW for 6 Gbps.

I. INTRODUCTION

Low-density parity-check (LDPC) codes first introduced by Gallager [1] have received considerable attention due to their excellent error-correction performance. Some recent technology standards, such as digital video broadcasting standard DVB-S2, and wireless local-area network standard IEEE 802.11n, have adopted LDPC codes. For their inherent parallelizable decoding algorithm, LDPC codes are also expected to be used for systems which require an even higher transmission rate. One such application is uncompressed video transmission using a millimeter waveband. To enable such applications, we designed a **rate-14/15 quasi-cyclic regular LDPC code** using a method previously reported [2], which is currently employed as a coding scheme in the draft document for the 60 GHz millimeter-wave-based wireless personal-area network standard IEEE 802.15.3c [3]. The rate 14/15, which may be the highest code rate for LDPC codes to date in communication standards, enables a throughput over 3 Gbps using quadrature phase shift keying (QPSK) modulation in a 2.16 GHz bandwidth.

Implementation of such a high throughput LDPC decoder with compact hardware using current LSI technology is a challenging task. The decoding algorithm iterates message-passing between two types of nodes, denoted as check nodes and variable nodes. The fully parallel decoder architecture can achieve very high throughput, however suffers

from inconveniently large hardware and interconnect complexity [4]. To reduce the amount of processing units, partially parallel decoder architectures have been proposed [5]–[7]. Because a check- or variable-node processing unit is usually shared by several check- or variable-node processes, the messages exchanged between the processing units have to be stored in memory. As a node processing unit requires messages generated by node processing units of the other type, message-passing between the two types of nodes cannot be processed simultaneously. Some methods have been reported to allow the processes to overlap by appropriate scheduling, which enables a throughput gain for some codes [6], [7].

When the throughput requirement is modest, the memory is efficiently implemented with a random-access memory (RAM). However, as the hardware needs to be parallelized for a higher throughput, a RAM with a large bit width and small depth would be area-inefficient. Meanwhile, implementing the memories with flip-flops (FFs) usually requires multiplexers and demultiplexers bridged with the processing units.

In this paper, we propose a simple encoding method for a quasi-cyclic code, and an LDPC decoding scheduling which improves the latency of overlapped message-passing. As the check-node operations are serialized, the decoder does not suffer from a large delay in the check-node calculation even for a large row weight. By utilizing the quasi-cyclic structure, composing memories used for the check-node processes of FFs with small additional circuits, which are connected cyclically, can eliminate the need for multiplexers and demultiplexers. We show the feasibility of an encoder and a decoder which can achieve a throughput over 6 Gbps with relatively small hardware for an LDPC code.

II. QUASI-CYCLIC LDPC CODES

In this section, the quasi-cyclic LDPC code is briefly reviewed. An (N, K) LDPC code of length N and dimension K is characterized by its $M \times N$ sparse parity-check matrix H . M rows in H correspond to constraints on the code. When every column of H has w_c non-zero elements and every row of H has w_r non-zero elements, the code is called a regular LDPC code, otherwise an irregular LDPC code. A class of quasi-cyclic code of length $N = pL$ is a linear block code such that every cyclic shift of a codeword by p symbols yields another codeword,

where p is a positive integer [8]. A general expression of a parity-check matrix H for a quasi-cyclic code can be written as

$$H = \begin{pmatrix} H_{0,0} & H_{0,1} & \cdots & H_{0,L-1} \\ H_{1,0} & H_{1,1} & \cdots & H_{1,L-1} \\ \vdots & \vdots & \ddots & \vdots \\ H_{q-1,0} & H_{q-1,1} & \cdots & H_{q-1,L-1} \end{pmatrix}, \quad (1)$$

where each $L \times p$ submatrix $H_{s,t}$ ($0 \leq s < q$, $0 \leq t < L$) is the cyclically t -row upward- or downward-shifted matrix of $H_{s,0}$. Every codeword $\mathbf{c} = [c_0 \ c_1 \ \dots \ c_{N-1}]^T$ satisfies M parity-check equations given as $H\mathbf{c} = \mathbf{0}$. We denote the set of variable positions that participate in check m by $N(m) = \{n \mid h_{m,n} \neq 0\}$, and the set of checks in which variable n participates as $M(n) = \{m \mid h_{m,n} \neq 0\}$, where $h_{m,n}$ ($0 \leq m < M$, $0 \leq n < N$) is an element in H . Fig. 1 shows an example of H for a binary (12,7) quasi-cyclic code with $q = 1$, $p = 2$, and $L = 6$.

A parity-check matrix given as an array of sparse circulants can be put into the form of (1) by permuting columns.

III. ENCODER

A. Encoding Algorithm for a Quasi-Cyclic Code

There is a method to encode a quasi-cyclic code in a form in which each p -bit section of a codeword is composed of $p-q$ information bits followed by q parity bits using a K -stage shift register [8]. But this method based on the generator matrix cannot be applied to a parity-check matrix whose rank $N-K$ is less than M . We propose here a new encoding method based on p generator polynomials for a quasi-cyclic code, which can also be applied to the case $N-K < M$. Using this encoding method, the information bits $[a_0 \ a_1 \ \dots \ a_{K-1}]$ appear at the first K -bit positions of a codeword, and the last $N-K$ bits are the parity bits $[r_0 \ r_1 \ \dots \ r_{N-K-1}]$. Denote the information polynomial $a(x)$, the parity polynomial $r(x)$, and the codeword polynomial $c(x)$ as

$$a(x) = \sum_{j=0}^{K-1} a_j x^{K-1-j}, \quad (2)$$

$$r(x) = \sum_{j=0}^{N-K-1} r_j x^{N-K-1-j}, \quad (3)$$

$$c(x) = a(x)x^{N-K} + r(x). \quad (4)$$

Now, we define p generator polynomials $g_0(x)$, $g_1(x)$, ..., $g_{p-1}(x)$, where each $g_j(x)$ ($0 \leq j < p$) is the codeword polynomial obtained by encoding the information polynomial $x^{(K+j) \bmod p}$, then the degree of $g_j(x)$ satisfies $\deg(g_j(x)) \bmod p = j$. Because of its quasi-cyclical nature, any codeword $c(x)$ can be expressed using the summation of p products as

$$c(x) = \sum_{j=0}^{p-1} g_j(x)q_j(x^p), \quad (5)$$

where each $q_j(x^p)$ is a polynomial of x^p . So encoding can be done by finding the parity polynomial which satisfies

$$r(x) = \sum_{j=0}^{p-1} g_j(x)q_j(x^p) - a(x)x^{N-K}. \quad (6)$$

B. Encoder Architecture Utilizing Quasi-Cyclic Structure

The encoding method described in Section III-A can be

$$H = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix}$$

$M = qL$ rows

Fig. 1. An example of a 6×12 parity-check matrix for a (12,7) quasi-cyclic code with $q = 1$, $p = 2$, and $L = 6$

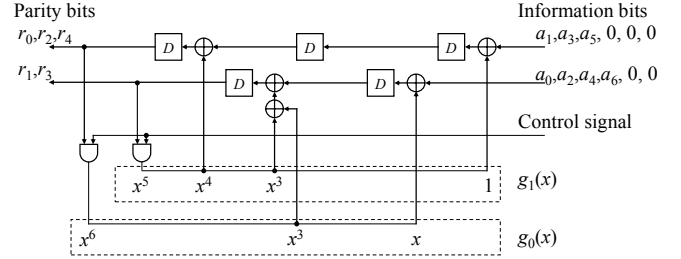


Fig. 2. Block diagram of a parity generator for the (12,7) quasi-cyclic code

implemented using $N-K$ registers and XOR circuits, similarly to that for a cyclic code.

Fig. 2 is a block diagram of a 2-bit parallel parity generator for the (12,7) quasi-cyclic code whose parity-check matrix is shown in Fig. 1. The coefficients in hexadecimal form of two generator polynomials derived by Gaussian elimination of the parity-check matrix are 4A for $g_0(x) = x^6 + x^3 + x$ and 39 for $g_1(x) = x^5 + x^4 + x^3 + 1$. Before encoding, all FFs are initialized to zeros. The control signal in Fig. 2 is activated while information bits including successive 5 zeros are input. The outputs are fed back to the positions corresponding to the coefficients of $g_0(x)$ and $g_1(x)$. Finally, the parity bits are obtained in the FFs.

IV. DECODER

A. Message-Passing Algorithm

The conventional message-passing algorithm for a binary LDPC code is summarized in this subsection. Let F_n be the log-likelihood ratio (LLR) of the n th variable received from the channel. Let $R_{m,n}^{(i)}$ and $Q_{m,n}^{(i)}$ denote the messages sent from check node m to variable node n , and from variable node n to check node m during the i th iteration, respectively. A posteriori LLR of the n th variable after the i th iteration is denoted as $Q_n^{(i)}$. The message-passing algorithm is expressed as follows.

Initialization step:

Set $i \leftarrow 1$,

$$Q_{m,n}^{(0)} = F_n \quad (7)$$

Step 1: Check-node process

$$R_{m,n}^{(i)} = \left(\prod_{n' \in N(m) \setminus \{n\}} \text{sign}(Q_{m,n'}^{(i-1)}) \right) \times \psi^{-1} \left(\sum_{n' \in N(m) \setminus \{n\}} \psi(Q_{m,n'}^{(i-1)}) \right) \quad (8)$$

Function $\psi(x) = \psi^{-1}(x) = -\ln(\tanh(x/2))$ defined for positive x is used for the sum-product algorithm.

Step 2: Variable-node process

$$Q_n^{(i)} = F_n + \sum_{m' \in M(n)} R_{m',n}^{(i)} \quad (9)$$

$$Q_{m,n}^{(i)} = F_n + \sum_{m' \in M(n) \setminus \{m\}} R_{m',n}^{(i)} \quad (10)$$

Step 3: If the hard decision $\hat{\mathbf{c}}^{(i)}$ of $Q_n^{(i)}$ satisfies $H\hat{\mathbf{c}}^{(i)} = \mathbf{0}$, or i reaches the maximum number of iterations $I_{\max}$, output $\hat{\mathbf{c}}^{(i)}$ as a decoded codeword. Otherwise, $i \leftarrow i+1$ and go to Step 1.

Usually, Step 1 and 2 are processed by check-node processing (CNP) units and variable-node processing (VNP) units, respectively. In a partially parallel decoder, each processing unit is shared by several node processes. The CNP units read $Q_{m,n}$ from memory with row-wise access, whereas the VNP units write $Q_{m,n}$ to memory with column-wise access, so the memory access control becomes complex.

The VNP unit which computes $Q_{m,n}$ for variable node n has to wait for all messages sent from check nodes $m' \in M(n)$. Similarly, the CNP unit which computes $R_{m,n}$ for check node m has to wait for all messages sent from variable nodes $n' \in N(m)$. This implies that the computations by the VNP units and the CNP units generally cannot be overlapped. Fig. 3(a) shows the timing chart where the CNP and VNP units operate alternately [5]. If no separate initialization step exists, multiplexers in the check-node processes are required in the first iteration.

To improve the throughput, overlapped message-passing decoding has been proposed [6], [7]. Fig. 3(b) shows the timing chart of the overlapped message-passing decoding. Waiting time w that avoids memory-access conflict can be minimized by appropriate scheduling, however, the achievable minimum waiting time w depends on the parity-check matrix. In Fig. 3(b), the separate initialization step can avoid memory conflict of multiple read accesses to F_n .

B. Scheduling of Message-Passing

In this subsection, scheduling to enable new overlapped message-passing is proposed. Let $A_m^{(i)}$ and $B_m^{(i)}$ ($0 \leq m < M$) be temporal variables for calculating $R_{m,n}$. The proposed steps for the initialization and the i th iteration are as follows.

Initialization step for the first iteration:

Set $i \leftarrow 1$, $A_m^{(1)} \leftarrow 1$, $B_m^{(1)} \leftarrow 0$,

For $0 \leq n < N$, $m \in M(n)$

$$Q_{m,n}^{(0)} = F_n \quad (11)$$

$$A_m^{(1)} \leftarrow A_m^{(0)} \times \text{sign}(Q_{m,n}^{(0)}) \quad (12)$$

$$B_m^{(1)} \leftarrow B_m^{(0)} + \psi(|Q_{m,n}^{(0)}|) \quad (13)$$

Step for the i th iteration:

Set $A_m^{(i+1)} \leftarrow 1$, $B_m^{(i+1)} \leftarrow 0$,

For $0 \leq n < N$, $m \in M(n)$

$$R_{m,n}^{(i)} = (A_m^{(i)} / \text{sign}(Q_{m,n}^{(i-1)})) \times \psi^{-1}(B_m^{(i)} - \psi(|Q_{m,n}^{(i-1)}|)) \quad (14)$$

$$Q_n^{(i)} = F_n + \sum_{m' \in M(n)} R_{m',n}^{(i)} \quad (15)$$

$$Q_{m,n}^{(i)} = F_n + \sum_{m' \in M(n) \setminus \{m\}} R_{m',n}^{(i)} \quad (16)$$

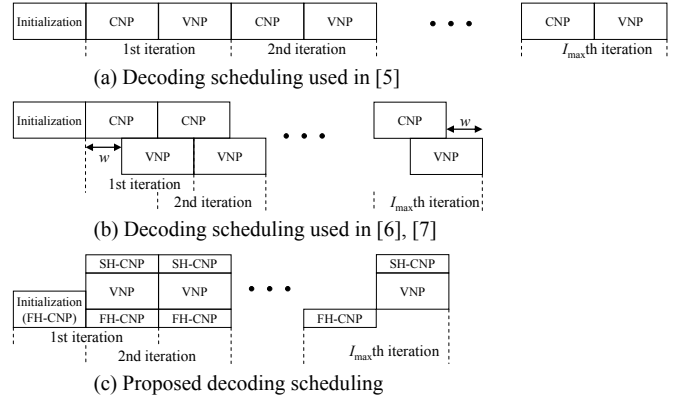


Fig. 3. Timing charts of CNP and VNP operations

$$A_m^{(i+1)} \leftarrow A_m^{(i)} \times \text{sign}(Q_{m,n}^{(i)}) \quad (17)$$

$$B_m^{(i+1)} \leftarrow B_m^{(i)} + \psi(|Q_{m,n}^{(i)}|) \quad (18)$$

Step 3 in Section IV-A follows the above steps.

In this scheduling, the check-node process is split into two processes, i.e. the first-half check-node process (FH-CNP) expressed by (17), (18) and the second-half check-node process (SH-CNP) expressed by (14), so that memory access is always column-wise. Equations (15), (16) are for the variable-node process. Immediately after the $R_{m,n}$ values are obtained in (14), the $R_{m,n}$ values are used to calculate the $Q_{m,n}$ values in (16), and the obtained $Q_{m,n}$ values are written to the $Q_{m,n}$ memory. The read access for (14) from the $Q_{m,n}$ memory becomes column-wise, just as the write access for (16) to the $Q_{m,n}$ memory. The obtained $Q_{m,n}$ values are also used to calculate $A_m^{(i+1)}$ and $B_m^{(i+1)}$ by (17), (18) which are prepared for use in the next iteration. The FH-CNP for the first iteration is executed in advance during the initialization step as (12), (13).

Fig. 3(c) shows the timing chart of this scheduling. As memory conflict does not occur because all memory accesses are column-wise, overlapped message-passing can be realized for any parity-check matrix.

It should be noted that the bit-error performance obtained by this decoding scheduling is equivalent to that obtained by the message-passing algorithm described in Section IV-A.

C. Decoder Architecture Utilizing Quasi-Cyclic Structure

Fig. 4 shows a block diagram of the proposed LDPC decoder. The LDPC decoder consists of an F_n memory, a $Q_{m,n}$ memory, N_p message-processing units, an A_m update unit, a B_m update unit, and a syndrome-check unit, where N_p is the degree of parallelization defined as the number of variable-node operations simultaneously executed. We assume that N_p is a multiple of p , and $d := N_p/p$ is a divisor of L .

The F_n memory in Fig. 4 stores two entire codeword blocks of the received LLRs for continuous decoding. While a codeword block is iteratively decoded, the incoming channel LLRs of the succeeding codeword block are just stored. The memory is configured such that the bit width for each read access is $N_p \times b_F$, where b_F is the bit width for each quantized F_n value. The F_n memory can be implemented with 2 single-port RAMs or 2 shift registers.

The $Q_{m,n}$ memory in Fig. 4 stores the variable-to-check messages $Q_{m,n}$. The size of the $Q_{m,n}$ memory is $N \times w_c \times b_Q$ bits, where b_Q is the bit width for each quantized $Q_{m,n}$ value. The $Q_{m,n}$ values obtained in the previous iteration are read by the message-processing units for calculating (14), and the updated $Q_{m,n}$ values obtained by (16) are written to the same address. The bit width for each access is $N_p \times w_c \times b_Q$. The $Q_{m,n}$ memory can be implemented with a dual-port RAM, a first-in first-out memory, or a shift register.

The syndrome-check unit in Fig. 4 checks that the syndrome vector $H\hat{c}^{(i)}$ is zero. This unit can be a similar structure to the A_m update unit described later.

Fig. 5 is a block diagram of the message-processing unit in Fig. 4, which is composed of w_c SH-CNP units, a VNP unit, and w_c FH-CNP units. The SH-CNP unit computes the check-to-variable message $R_{m,n}$ using (14). The VNP unit receives a channel LLR F_n and w_c check-to-variable messages $R_{m,n}$, and computes the hard decision $\hat{c}_n$ of *a posteriori* LLR Q_n and w_c variable-to-check messages $Q_{m,n}$, at one time. The FH-CNP unit computes the sign and ψ function of (17) and (18). The number of output bits of the ψ function is denoted as b_ψ . The initialization step can be done by substituting zeros for the outputs from the SH-CNP units. The ψ functions in the SH-CNP units may be omitted when the output values from the FH-CNP units instead of from the VNP units are stored in the $Q_{m,n}$ memory. But the memory size must be increased because $\psi(x)$ is a rapidly decreasing function for small positive x .

The A_m update unit in Fig. 4 consists of 2 sets of M -bit memories and 1-bit multiplication circuits. The B_m update unit in Fig. 4 consists of 2 sets of $(M \times b_B)$ -bit memories and adder circuits, where b_B is the bit width for each quantized B_m value. The reason 2 sets of memories are used is to enable concurrent operations of the SH-CNP for the i th iteration and the FH-CNP for the $(i+1)$ -th iteration. The A_m and B_m update unit outputs $A_m^{(i)}$ and $B_m^{(i)}$ values held in one set of memories to $N_p \times w_c$ SH-CNP units during N/N_p cycles. The number of different indices of A_m and B_m required by the SH-CNP units is denoted as N_A . During the N/N_p cycles, the A_m and B_m update unit receives $\text{sign}(Q_{m,n}^{(i)})$ and $\psi(|Q_{m,n}^{(i)}|)$ from $N_p \times w_c$ FH-CNP units, and the values in the other set of memories are updated according to (17) and (18). After the N/N_p cycles, $A_m^{(i+1)}$ and $B_m^{(i+1)}$ are obtained. When a large N_p value is employed, serial multiplications of (17) and serial additions of (18) are required because messages from multiple variable nodes are received.

When the parity-check matrix is not well structured, the A_m and B_m update units require complex multiplexers and demultiplexers. We introduce hardware architecture here which can eliminate them when a quasi-cyclic code is used.

Suppose $h_{0,0} = 1$, and $H_{s,t}$ is defined as the cyclically t -row downward-shifted matrix of $H_{s,0}$. Then N/N_p elements $h_{m,n}$ are 1 for $[m \ n] \in \{[kd \ kN_p] \mid 0 \leq k < N/N_p\}$. When each memory location in the A_m update unit has a one-to-one correspondence to a check-node index, the memory location for the check-node index kd varies depending on k . When the memories in the A_m update unit are connected to form cycles of length L/d , the memory location for the check-node index kd can be fixed for

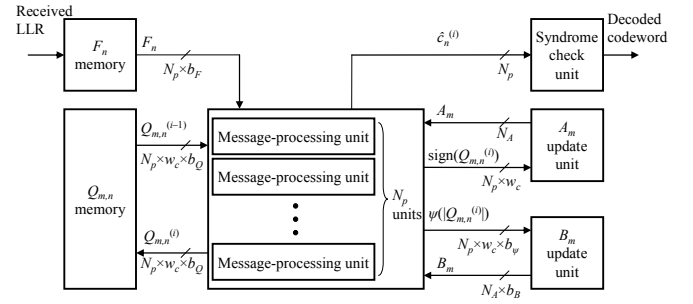


Fig. 4. Block diagram of the LDPC decoder

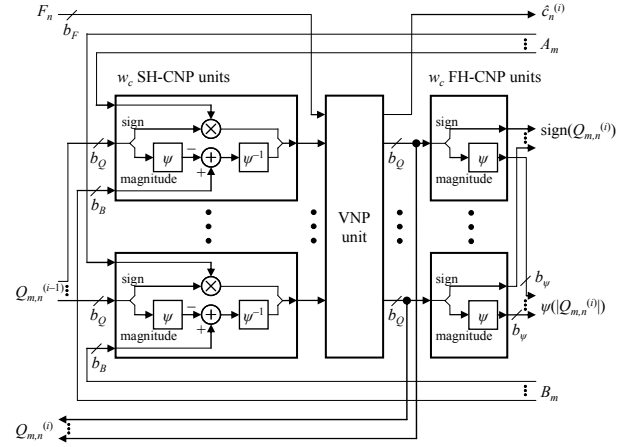


Fig. 5. Block diagram of the message-processing unit in Fig. 4

every k because of the quasi-cyclic structure.

The A_m and B_m update units utilizing the quasi-cyclic structure are similarly configured using M subunits $U_0, U_1, \dots, U_{M-1}$. The M subunits can be grouped into qd disjoint sets, each of which consists of L/d subunits $\{U_{sL+u+vd} \mid v = 0, 1, 2, \dots, L/d-1\}$, where $0 \leq s < q$ and $0 \leq u < d$. Fig. 6 shows the block diagram representing a set of L/d subunits, where the bit width b_U is 1 for A_m and b_B for B_m . The subunits in a set form a cycle.

Every subunit in the A_m update unit contains 2 FFs. Fig. 7(a) and (b) show the possible configurations for a subunit in the A_m update unit. Fig. 7(a) is used for a subunit $U_{sL+u+vd}$ if all the N_p elements $h_{sL+u+vd,x}$ are 0 for $0 \leq x < N_p$, otherwise Fig. 7(b) is used. At time index k , the subunit $U_{sL+u+vd}$ of Fig. 7(b) outputs $A_m^{(i)}$, where $m = sL + \{(u+vd+kd) \bmod L\}$, and signs of $Q_{m,n}^{(i)}$ are input, where n takes kN_p+x for all x which satisfy $h_{sL+u+vd,x} = 1$ for $0 \leq x < N_p$. During the i th iteration, one of the two FFs contains $A_m^{(i)}$ value and the other is used for an intermediate value to obtain $A_m^{(i+1)}$. The contents in the FFs for $A_m^{(i)}$ are discarded at the end of the i th iteration, and the FFs are used for $A_m^{(i+2)}$ during the $(i+1)$ -th iteration. The control signal for 2:1 multiplexers in Fig. 7(b) indicates whether the iteration number i is odd or even.

The B_m update unit is similar to the A_m update unit, but each subunit includes $2 \times b_B$ FFs. Fig. 7(c) is substituted for (b). At time index k , the subunit $U_{sL+u+vd}$ of Fig. 7(c) outputs $B_m^{(i)}$, where $m = sL + \{(u+vd+kd) \bmod L\}$, and values of $\psi(|Q_{m,n}^{(i)}|)$ are input, where n takes kN_p+x for all x which satisfy $h_{sL+u+vd,x} = 1$ for $0 \leq x < N_p$.

Pipeline operation is possible by inserting registers in the message-processing units. The total number of clock cycles for

$I_{\max}$ decoding iterations is $(L/d+N_R)(I_{\max}+1)$, where N_R is the number of pipeline stages. When the decoder operates at frequency f , it can decode continuously received blocks at the bit rate T_{in} given by $Nf/\{(L/d+N_R)(I_{\max}+1)\}$.

This decoder architecture can be applied to an irregular LDPC code by scaling each message-processing unit to its corresponding column weight, and can also be applied to shuffled decoding [9] to reduce the number of iterations with a slight modification of the A_m and B_m update units, although these applications are not discussed here.

V. DESIGN OF A (1440, 1344) LDPC ENCODER AND DECODER

We have designed practical LDPC encoders and decoders adopting the architecture described in the previous sections. All designs were synthesized using a 65 nm CMOS technology library under the condition of a temperature of 125 °C and a voltage of 1.08 V. The gate count was obtained by adding the number of gates for all cells, where a gate is defined as four transistors. The actual chip was not available, so the power consumption was an estimation using the number of gates, operating frequency, and a roughly assumed switching activity.

The LDPC code employed in this paper is the rate-14/15 quasi-cyclic regular LDPC code designed for IEEE 802.15.3c with $N=1440$, $K=1344$, $p=15$, $q=1$, $L=96$, $w_c=3$, $w_r=45$, and the minimum Hamming distance of 6. We chose a regular LDPC code, whose decoder could be simpler than that of an irregular LDPC code. Note that the threshold advantage of an irregular LDPC code over a regular LDPC code is at most 0.3 dB for a high code rate above 0.9 [10]. Furthermore, the advantage in coding gain vanishes when the codeword length is short enough to be practical for a high-rate code. For a communication system, a low-rate code is important for high reliability, and so is a high-rate code for high bandwidth efficiency. As the data rate of the uncompressed video of (1920×1080) pixels $\times$ 24 bits per pixel $\times$ 60 Hz is as high as 3 Gbps, the code rate should be high enough that the video can be transmitted even using simple modulation within a limited bandwidth. We set the target throughput to 3.2256 Gbps using the rate-14/15 LDPC code, i.e. the coded bit rate of 3.456 Gbps, at which data can be transmitted using QPSK with a symbol rate of 1.728 G symbols per second. Table I lists indices $[m\ n]$, which give the locations whose elements $h_{m,n}$ in the submatrix $H_{0,0}$ are 1. Other submatrices $H_{0,t}$ are obtained by cyclically t -row upward shifting of $H_{0,0}$.

A 15-parallel encoder was designed using the method described in Section III. Table II shows the coefficients of 15 generator polynomials in hexadecimal form. The hardware size including additional 96 FFs for continuous operation was 2.5 k gates when the operating frequency is 230.4 MHz. The encoder can process input data at 3.2256 Gbps. Power consumption was estimated to be 2.0 mW for 1.2 V supply, assuming the switching activity is 0.5. A 30-parallel encoder for 6.4512 Gbps was also designed. Its gate count was 3.1 k.

As for the LDPC decoder, the higher the degree of parallelization N_p , the higher the throughput the decoder is

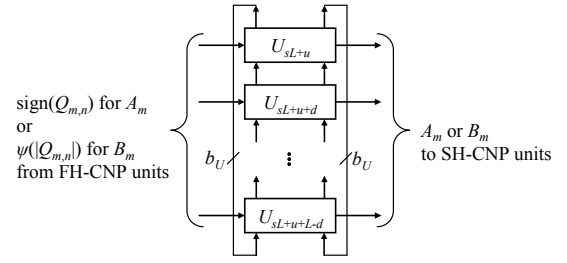


Fig. 6. Block diagram of a set of L/d subunits U_{sL+u+v} utilizing quasi-cyclic structure used for the A_m and B_m update units in Fig. 4

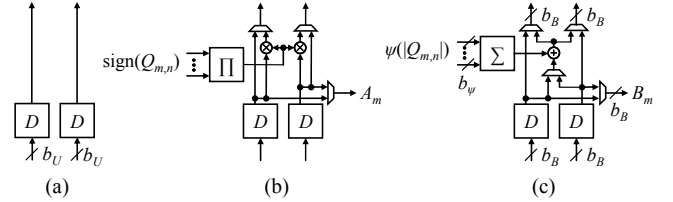


Fig. 7. Block diagrams of possible configurations for a subunit $U_{sL+u+vd}$ in Fig. 6 (a) for the case all the elements $h_{sL+u+vd,x}$ are 0 for $0 \leq x < N_p$, (b) for a case other than (a) for the A_m update unit, and (c) for a case other than (a) for the B_m update unit

TABLE I. PARAMETERS TO DEFINE THE (1440, 1344) LDPC MATRIX

[0 0]	[1 0]	[4 0]	[32 1]	[34 1]	[39 1]
[64 2]	[70 2]	[78 2]	[8 3]	[18 3]	[95 3]
[31 4]	[42 4]	[54 4]	[63 5]	[76 5]	[91 5]
[14 6]	[45 6]	[94 6]	[30 7]	[47 7]	[83 7]
[17 8]	[62 8]	[80 8]	[28 9]	[48 9]	[82 9]
[22 10]	[60 10]	[81 10]	[27 11]	[49 11]	[84 11]
[7 12]	[53 12]	[77 12]	[19 13]	[44 13]	[85 13]
[6 14]	[46 14]	[75 14]			

TABLE II. GENERATOR POLYNOMIALS FOR THE (1440, 1344) LDPC CODE

$g_0(x)$	02008A43C438280AD3220414CA2A
$g_1(x)$	04002A09100310379125214082C6
$g_2(x)$	080000D28CCD0C1810880440132
$g_3(x)$	10000D0C04900416612A24326480
$g_4(x)$	2000020C28F089055000E41448A4
$g_5(x)$	4000C00EC83E99084209444488C
$g_6(x)$	0001C715CC84A50CF30800466488
$g_7(x)$	00021D6C08806496E3284403E684
$g_8(x)$	00044F0A2C76C4827000C003AE26
$g_9(x)$	0008228098021139022FA0140069
$g_{10}(x)$	00104A04840E4D8A702200578618
$g_{11}(x)$	00206406B47A41CDC321844049AA
$g_{12}(x)$	0040CD0700BE4D87A31844017E24
$g_{13}(x)$	00801460E64A7198222BB21600A8
$g_{14}(x)$	0100C909A4C60C1F520AE010A62E

expected to achieve for its high gate count and its complexity. The number $d = N_p/p$ should be a divisor of L , so the available number N_p should be either 15, 30, 45, 60, 90, 120, 180, 240, 360, 480, or 720. We adopted $N_p = 180$ for 3.2256 Gbps, and $N_p = 360$ for a higher throughput of 6.4512 Gbps.

The maximum operating frequency is determined by the delay of an adder in the B_m update unit. In the case of $N_p = 360$, messages from 360 variable nodes are processed in the B_m update unit simultaneously. Because the maximum row weight of the first 360 columns of H is 15, the worst delay is caused by the 15 serial evaluations of (18). An adder with 15+1 inputs operates faster than the adder with $w_r = 45$ inputs which is required when a parallel CNP unit is used. As was pointed out by Mansour *et al.* in [11], for large w_r , the parallel implementation of the CNP unit becomes too complex and serial implementation degrades the throughput. Though our approach serializes the check-node process, the throughput is

not degraded because of overlapped message-passing.

The VNP unit for $w_c = 3$ was realized using the 2-stage cascaded adders for calculating (15), (16) as is shown in [12].

Fig. 8 shows the ratio of required bit energy to noise power spectral density (E_b/N_0) to achieve the bit-error rate (BER) of 10^{-6} obtained using simulation, when binary phase shift keying modulation on the additive white Gaussian noise channel is assumed. At least 100 codeword errors were observed for each simulated E_b/N_0 . The required E_b/N_0 for $I_{\max} = 8$ is 5.9 dB when the bit widths for quantized values are $b_F = 4$, $b_Q = 3$, $b_\psi = 6$, and $b_B = 7$. Loss induced by quantization was within 0.07 dB. Fig. 9 shows the BER performance for $I_{\max} = 8$. The average number of iterations I_{ave} was 1.54 at $E_b/N_0 = 5.9$ dB.

A 180-parallel decoder with 1-stage pipeline registers was synthesized with an operating frequency of 194.4 MHz. The total gate count of the decoder was 304 k. The decoder can operate at a coded bit rate T_{in} of 3.456 Gbps for $I_{\max} = 8$. The power consumption was estimated to be 58 mW for 1.2 V supply, assuming the switching activity is 0.5 for $(I_{\text{ave}}+1)/(I_{\max}+1) = 28\%$ of the entire duration and 0 for the rest. The power consumption will be reduced for a higher E_b/N_0 value because the average number of iterations becomes smaller. A 360-parallel decoder with 1-stage pipeline registers operating at 216 MHz doubled the throughput of the 180-parallel decoder. The gate count of the 360-parallel decoder was 409 k. The 360-parallel decoder is a possible choice even for 3.2256 Gbps, because 26 % lower power consumption compared with that for the 180-parallel decoder is expected for its 39 % larger cell area when operated at 108 MHz. In [13], a high throughput decoder of 5.92 Gbps with 8 iterations for a similar codeword length $N = 1200$ using 0.13 μm CMOS technology was reported. Our decoder achieves 1.09 times this throughput with 0.36 times the gate count of [13], although we employed advanced CMOS technology.

Table III shows a summary of our designed hardware. All the designs were also synthesized using a 90 nm CMOS technology library at the same frequency. The gate count of each design was almost the same as when using 65 nm, but the cell area and power consumption were approximately doubled.

ACKNOWLEDGMENT

The authors would like to thank K. Kondou for fruitful discussions on LDPC codes. The authors would also like to thank K. Kawasaki and Dr. K. Fukuzawa for discussions on the millimeter wave applications and supporting this study.

REFERENCES

- [1] R. G. Gallager, "Low-density parity-check codes," IRE Trans. Inform. Theory, vol. IT-8, pp. 21-28, Jan. 1962.
- [2] M. Noda, "Designing a self-orthogonal quasi-cyclic code with extended minimum hamming distance," Proc. 4th Int. Sympo. Turbo Codes and Related Topics, Apr. 2006.
- [3] H. Harada, et al., "SC-PHY MCS updates of the baseline document," IEEE P802.15 Working Group for Wireless Personal Area Networks, IEEE802.15-08-0056-01-003c, Jan. 2008.
- [4] A. J. Blanksby and C. J. Howland, "A 690-mW 1-Gb/s 1024-b, rate-1/2 low-density parity-check code decoder," IEEE J. Solid-State Circuits, vol. 37, no. 3, pp. 404-412, Mar. 2002.

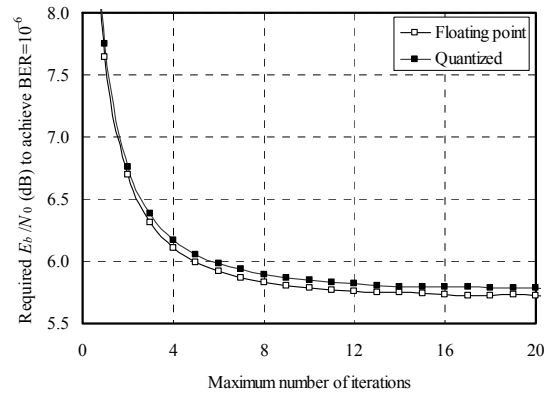


Fig. 8. Required E_b/N_0 to achieve a bit-error rate of 10^{-6} as a function of the maximum number of iterations

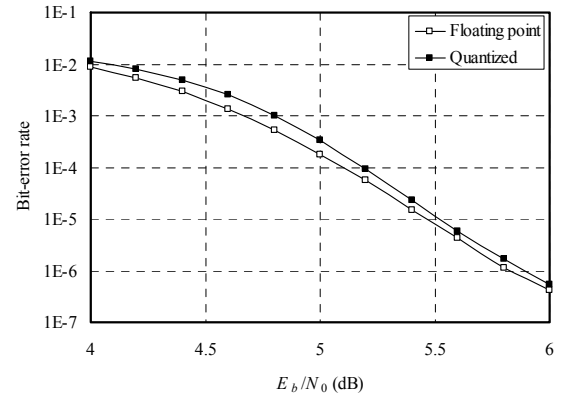


Fig. 9. Bit-error rate performance as a function of E_b/N_0 when the maximum number of iterations is 8

TABLE III. SUMMARY OF HARDWARE OF THE LDPC ENCODER AND DECODER

	Encoder		Decoder ($I_{\max} = 8$)	
Throughput (Gbps)	3.2256	6.4512	3.2256	6.4512
Degree of parallelization	15	30	180	360
Cell area (mm^2)	0.0030	0.0037	0.36	0.50
Gate count	2.5 k	3.1 k	304 k	409 k
Operating frequency (MHz)	230.4	230.4	194.4	216
Power consumption est. (mW)	2.0	2.4	58	86

- [5] T. Zhang and K. K. Parhi, "A 54 Mbps (3,6)-regular FPGA LDPC decoder," Proc. SIPS 2002, pp. 127-132, Oct. 2002.
- [6] Y. Chen and K. K. Parhi, "Overlapped message passing for quasi-cyclic low-density parity-check codes," IEEE Trans. Circuits and Systems I, vol. 51, no. 6, pp. 1106-1113, Jun. 2004.
- [7] I.-C. Park and S.-Y. Kang, "Scheduling algorithm for partially parallel architecture of LDPC decoder by matrix permutation," Proc. ISCAS 2005, pp. 5778-5781, May 2005.
- [8] R. L. Townsend and E. J. Weldon, Jr., "Self-orthogonal quasi-cyclic codes," IEEE Trans. Inform. Theory, vol. IT-13, no. 2, pp. 183-195, Apr. 1967.
- [9] J. Zhang and M. P. C. Fossorier, "Shuffled iterative decoding," IEEE Trans. Commun., vol. 53, no. 2, pp. 209-213, Feb. 2005.
- [10] S.-Y. Chung, T. J. Richardson, and R. L. Urbanke, "Analysis of sum-product decoding of low-density parity-check codes using a Gaussian approximation," IEEE Trans. Inform. Theory, vol. 47, no. 2, pp. 657-670, Feb. 2001.
- [11] M. M. Mansour and N. R. Shanbhag, "High-throughput LDPC decoders," IEEE Trans. VLSI systems, vol. 11, no. 6, pp. 976-996, Dec. 2003.
- [12] Y. Li, M. Elmassal, and M. Bayoumi, "Power efficient architecture for (3,6)-regular low-density parity-check code decoder," Proc. ISCAS 2004, vol. 4, pp. 81-84, May 2004.
- [13] C.-C. Lin, K.-L. Lin, H.-C. Chang, and C.-Y. Lee, "A 3.33Gb/s (1200,720) low-density parity check code decoder," Proc. ESSCIRC 2005, pp. 211-214, Sept. 2005.